

Dobot X-Trainer（轻量版）

用户手册

文档版本：V1.3

发布日期：2024-07-29

版权所有 © 越疆科技股份有限公司2024。 保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

免责声明

在法律允许的最大范围内，本手册所描述的产品（含其硬件、软件、固件等）均“按照现状”提供，可能存在瑕疵、错误或故障，越疆不提供任何形式的明示或默示保证，包括但不限于适销性、质量满意度、适合特定目的、不侵犯第三方权利等保证；亦不对使用本手册或使用本公司产品导致的任何特殊、附带、偶然或间接的损害进行赔偿。

在使用本产品前详细阅读本使用手册及网上发布的相关技术文档并了解相关信息，确保在充分了解机械臂及其相关知识的前提下使用机械臂。越疆建议您在专业人员的指导下使用本手册。该手册所包含的所有安全方面的信息都不得视为Dobot的保证，即便遵循本手册及相关说明，使用过程中造成的危害或损失依然有可能发生。

本产品的使用者有责任确保遵循相关国家的切实可行的法律法规，确保在越疆机械臂的使用中不存在任何重大危险。

越疆科技股份有限公司

地址：深圳市南山区留仙大道3370号南山智园崇文区2号楼1003

网址：www.dobot.cn

前言

目的

本手册用于指导用户安装和配置Dobot X-Trainer AI机器人操作平台（轻量版）。

读者对象

本手册适用于：

- 客户
- 销售工程师
- 安装调试工程师
- 技术支持工程师

修订记录

时间	版本号	修订记录
2024/07/29	V1.3	优化环境配置和算法训练相关的说明
2024/07/12	V1.2	1. 区分两个从手夹爪和相机的安装方式 2. 环境配置和遥操作操作步骤细节优化 3. 训练和推理的episode_len参数说明优化
2024/06/11	V1.1	1. 增加相机连接的注意事项和相机测试的脚本说明 2. 增加主手垫高底座的说明
2024/05/22	V1.0	第一次发布

符号约定

在本手册中可能出现下列标志，它们所代表的含义如下。

符号	说明
 危险	表示有高度潜在危险，如果不能避免，会导致人员死亡或严重伤害
 警告	表示有中度或低度潜在危害，如果不能避免，可能导致人员轻微伤害、机械臂毁坏等情况
 注意	表示有潜在风险，如果忽视这些文本，可能导致机械臂损坏、数据丢失或不可预知的结果
 说明	表示是正文的附加信息，是对正文的强调和补充

目 录

前言	ii
1. 产品介绍	1
1.1 概述	1
1.2 产品组成	2
1.3 电控柜接口说明	3
2. 硬件安装	4
2.1 安装环境	4
2.2 开箱	4
2.3 安装步骤	5
2.3.1 整体布局	5
2.3.2 安装电控柜	5
2.3.3 安装从手	6
2.3.4 安装急停开关	7
2.3.5 安装主手	7
2.3.6 安装全局相机	8
2.3.7 安装从手夹爪及相机	11
2.3.8 连接指示灯	13
2.3.9 连接训练主机	14
2.3.10 整理线缆	14
2.3.11 设备上电	15
3. 运行环境配置	17
3.1 操作系统	17
3.2 安装 CUDA 和 cuDNN	17
3.3 安装 Anaconda	18
3.4 配置虚拟环境	19
3.5 从手设置	21
3.5.1 修改从手 2 的 IP	21
3.5.2 配置从手 1 和 2	23
3.6 配置训练主机 IP	25
4. 遥操作与数据采集	26
4.1 主手操作说明	26
4.2 配置并采集数据	26
5. 模仿学习算法训练	31
5.1 训练数据集处理	31

5.2 设置数据集相关参数.....	31
5.3 设置训练相关参数.....	33
6. 自主推理运行	35
7. 数据接口说明	36

1. 产品介绍

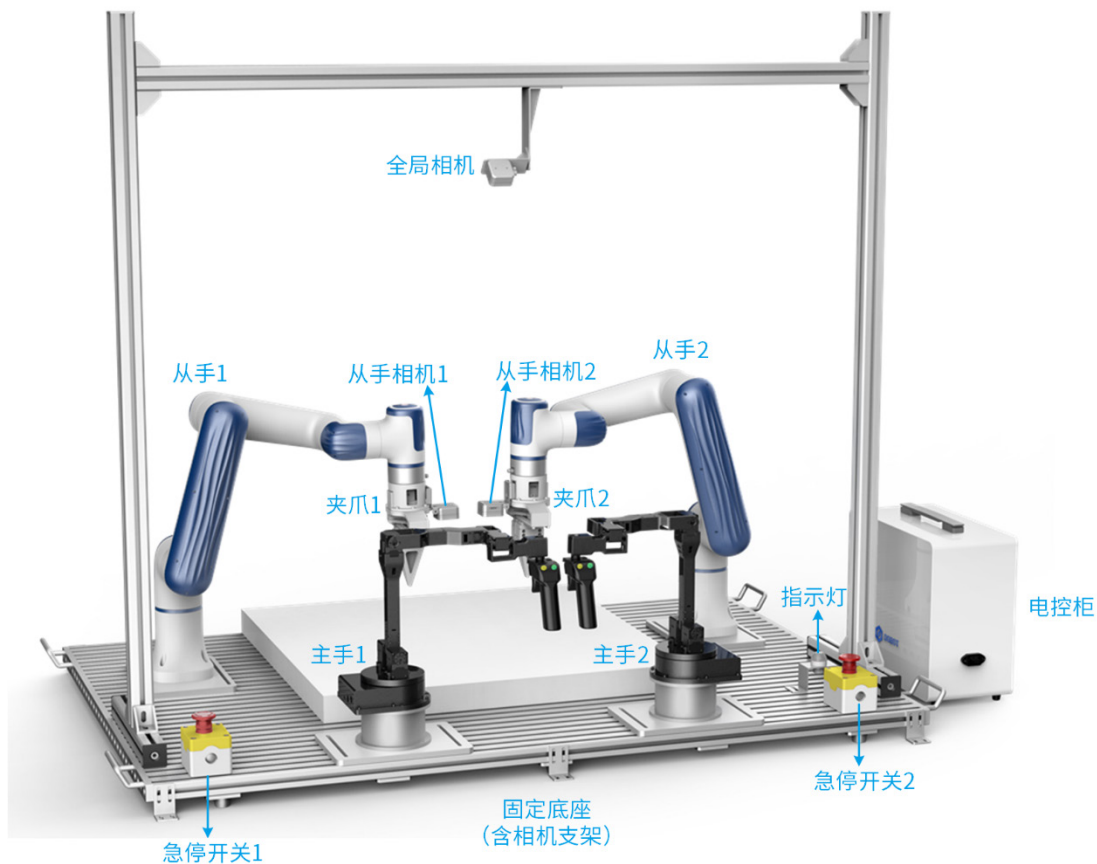
1.1 概述

当前国内外对通用人工智能（AGI）的研究进入新的热潮，具身智能（Embodied AI）是迈向通用人工智能（AGI）的重要一步。开发具身智能一个非常有前景方法是，从人类提供的演示中进行模仿学习。这种「行为克隆」可以让机器人学习各种原始的技能，从简单的拾放操作，到更精细的操作等等。

本套件提供了一个AI机器人操作平台，通过直接操纵小型主手来直观、灵巧和精确地控制从手机械臂运动，实现可靠、高效、高质量的演示数据采集。同时提供了一个基础的模仿学习数据采集、算法训练与推理的示范代码程序、以及整套平台的API接口（包含主手关节角度数据读取、主手锁定控制、从手机械臂关节角度和笛卡尔坐标读取/控制、夹爪开合度控制、相机RGB/深度图像获取等接口）。

通过本硬件平台和二次开发接口，可实现灵活、丰富的机器人应用研究。例如家庭服务机器人，帮助人们完成日常家务，如烹饪、清洁和整理；工业自动化机器人，完成装配和质量检测等任务。

1.2 产品组成



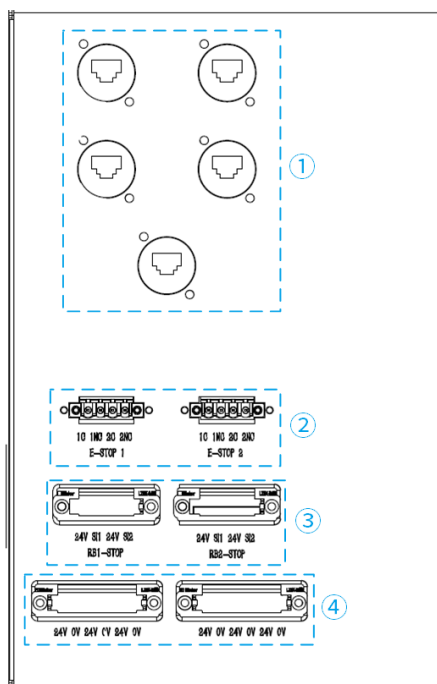
- **主手**：6自由度高精度操作主手，左右各一个，末端带如下图的操作手柄。



- **从手**：Nova 2机械臂（控制器版本3.5.7及以上），左右各一个。与主手同步后会跟随主手进行运动。
- **夹爪**：两个从手末端各一个。最大开合度95mm。与主手同步后通过主手末端的扳机进行控制。
- **相机**：Intel RealSense D405相机，共三个。相机支架上安装一个作为全局相机，两个从手末端各安装一个作为从手相机。

- **急停开关：**从手的急停开关，拍下任一个急停按钮都会使两个从手紧急制动。
- **指示灯：**三色指示灯，含义如下。
 - 灭灯：主从手未同步。
 - 黄灯：主从手已同步，未录制数据。
 - 绿灯：录制中。
 - 红灯：报错等异常状态。
- **电控柜：**电气控制柜，使用市电作为电源输入，背面接口详见[电控柜接口说明](#)。
- **固定底座：**X-Trainer的安装底座，包含安装全局相机的相机支架。

1.3 电控柜接口说明



- ①：5个网口，用于连接从手控制器（CCBOX）和电脑。
- ②：2个急停开关接口。
- ③：2个机器人急停输出接口，用于连接从手控制器的急停输入接口。
- ④：2排（共6组）24V输出接口，主要用于给从手夹爪供电。

2. 硬件安装

2.1 安装环境

为了维持X-Trainer的性能并确保安全使用，请将X-Trainer放置在符合以下条件的环境中。

- 请安装于室内，且通风良好。
- 请勿在过度振动和冲击的环境下安装和使用。
- 应避免阳光照射，不涉及辐射热。
- 空气中无灰尘、油雾、油烟、盐分、铁屑、腐蚀性气体等。
- 请勿在封闭环境中使用。封闭环境易导致控制柜高温，缩短使用寿命。
- 请勿在可燃物附近使用。
- 请勿在有切削液、磨削液的烟雾环境中使用。
- 附近无大型变压器、大型电磁接触器、电焊机等电磁干扰源。

2.2 开箱

开箱前请仔细检查外包装是否有在运输过程中损坏的迹象。如果发现任何损坏，请在拆箱时要求货运公司人员在场。

开箱时，请对照附带的产品清单，确保内容物无遗漏且无损坏。如有任何内容物缺失或损坏，请联系您的供应商。

开箱后，请妥善保存所有外包装，便于日后运输使用。

除X-Trainer标配的部件外，用户还需准备下述部件。

- 自备一台电脑，用于遥操作和算法训练（后文简称为**训练主机**），配置要求如下（如果仅进行遥操作，配置要求较低）：

配置项	推荐配置	
	仅遥操作	算法训练
显卡	GTX1050	RTX4090，24GB显存 (最低RTX2080 Ti，11GB显存)
CPU	Intel i5-9300H	Intel i9-14900KF
内存	16GB	32GB
硬盘	1TB	2TB SSD
操作系统	Ubuntu 20.04	

- 自备另一台电脑（Window 7/10/11操作系统，也可在训练主机上安装双系统）或平板（安卓10以上或iOS10以上），用于安装从手（Nova2）的控制软件（后文简称为**从手操作终端**），配置要求如下：

终端类型	PC	安卓平板	iPad
操作系统	Windows7/10/11	Android 10及以上	iOS 10及以上
控制软件	DobotStudio Pro	Dobot CRStudio	Dobot CRStudio
最低配置	CPU：Intel Core i3 内存：4G 硬盘空间：128GB 网卡：千兆网卡 显存：1G	CPU：4核 运行内存：2G 存储空间：32G 显示屏：8寸	iPad 5代及以上
通讯方式	LAN/WiFi	WiFi	WiFi

*如需使用WiFi通讯，请在购买X-Trainer时选配Nova的WiFi模块。

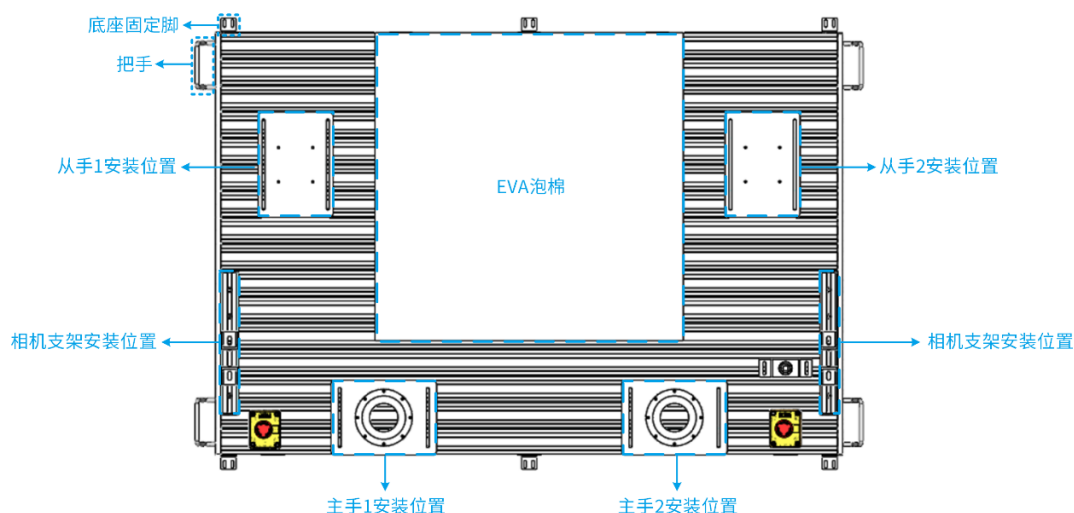
- 自备符合当地标准的排插，除电脑外，还需预留至少5个插孔给X-Trainer供电。

2.3 安装步骤

2.3.1 整体布局

轻量版X-Trainer底座尺寸为1400 x 1000mm（不包含把手），整体布局如下图所示。

中央的EVA泡棉为发货附件，请参考下图自行摆放。两侧的6个固定脚发货时为散件，如果客户需要将底座固定到台面上，请使用M5*10螺钉自行安装固定脚。



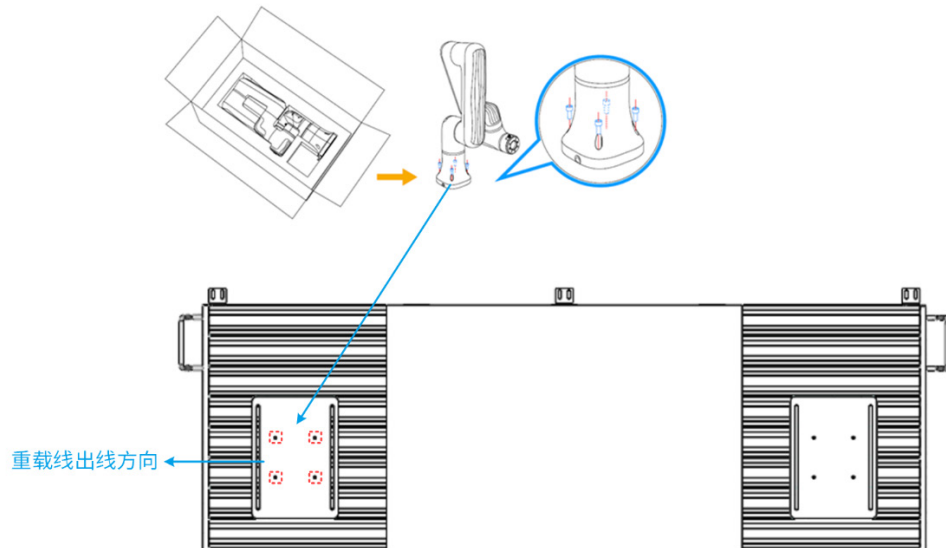
2.3.2 安装电控柜

从包装中取出电控柜，放置在底座旁边。电控柜相关接线请参考后文。

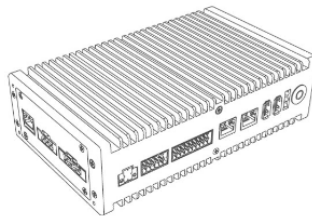
2.3.3 安装从手

所需工具：内六角扳手*2（M3/M6），M3*8螺钉若干。

1. 从从手1安装板上拆下4个M6内六角螺钉（下图中红框），将一台从手（Nova 2）从包装中取出，使用刚才拆下的螺钉固定到安装板上，**重载线朝向左侧**。从包装箱中将机器人移至安装位置时，请扶住机器人直至机器人底座的所有螺栓全部紧固好。



2. 将这台从手配套的控制器（CCBOX，如下图所示）放到从手附近，每侧应留出至少50mm的空隙，保证足够的散热空间。



3. 参考上述步骤将另一台从手安装到从手2安装位置，**重载线朝向右侧**，两台从手呈镜像对称。
4. 参考下图，连接从手相关电缆（两个从手的连接方式相同）。

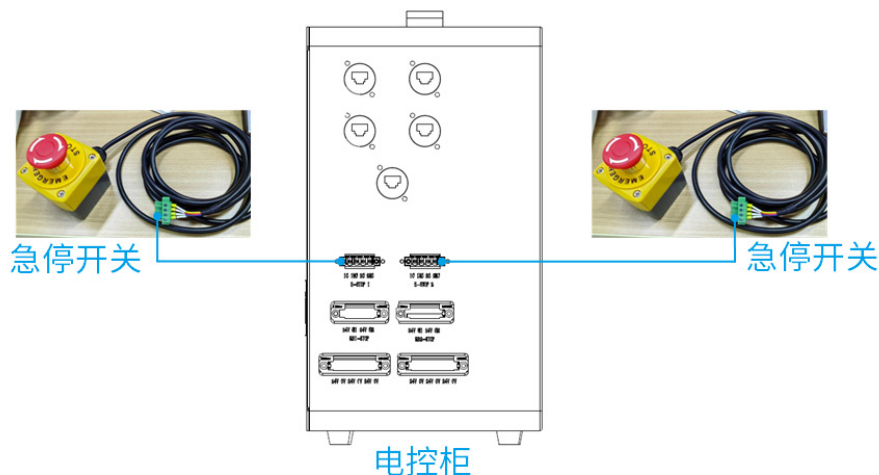


RB-STOP线如下图，黑色端子接CCBOX，绿色端子接电控柜。



2.3.4 安装急停开关

将急停开关放置在安装底座上，线缆连接至电控柜的急停开关接口。

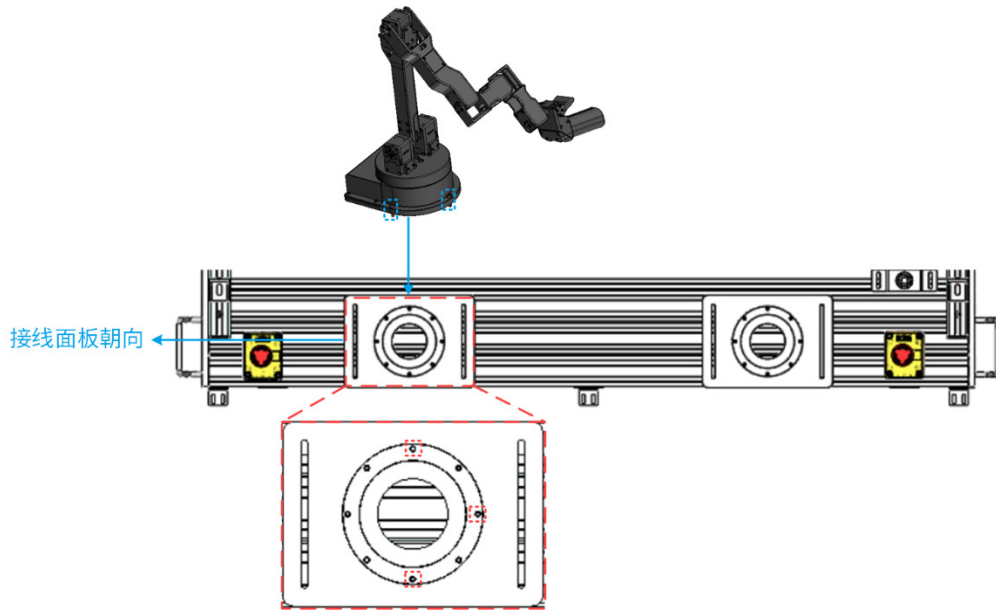


2.3.5 安装主手

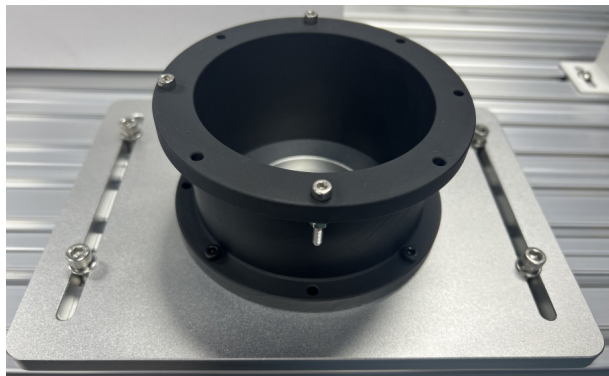
所需工具：内六角扳手*1（M5）。

1. 从主手1安装底座上拆下3个M5内六角螺钉（下图中红框），将一台主手从包装中取

出，使用刚才拆下的螺钉固定到安装板上，**接线面板朝向左侧**。



主手安装板上默认安装了如下图所示的圆柱形底座，可根据现场需求（例如操作者坐高较低）将其拆除，将主手直接安装到下方的安装板上。



2. 参考上述步骤将另一台主手安装到主手2安装位置，**接线板朝向右侧**，两台主手呈镜像对称。
3. 参考下图，连接主手的电源线和数据线（两个主手的连接方式相同）。



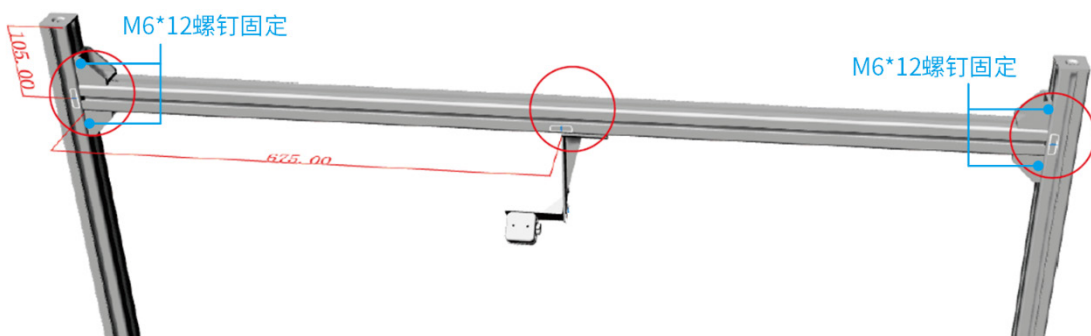
2.3.6 安装全局相机

所需工具：内六角扳手*2（M4/M6），M6*12螺钉/M6*8螺钉/M6平垫片若干。

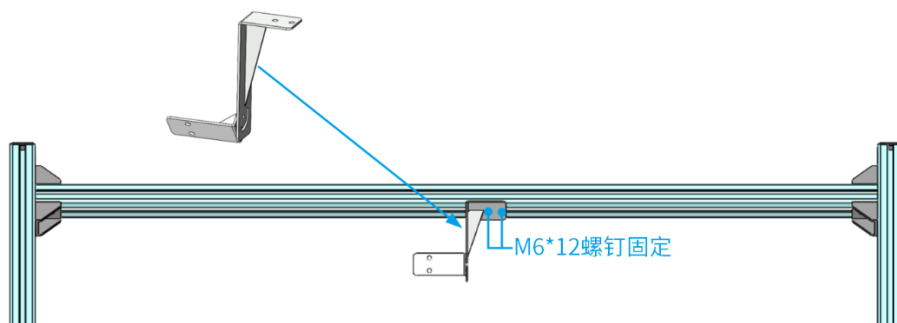
安装相机前，建议先记录各个相机的ID（如下图）和安装位置，后续配置时需要。



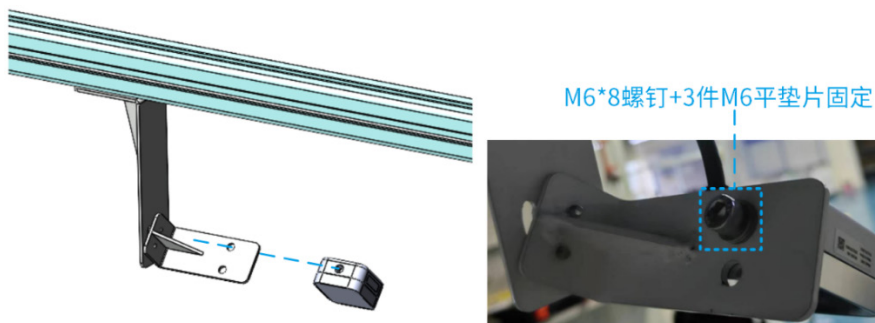
1. 相机支架包括两根竖杆和一根横杆，竖杆长1.2m，横杆长1.32m。使用4颗M6*12螺钉组装横杆和竖杆，组装时请参考下图对准贴纸的位置。横杆和竖杆内均内置型材螺母，组装时请将型材螺母滑到对应位置用于固定螺钉，后文不再赘述。



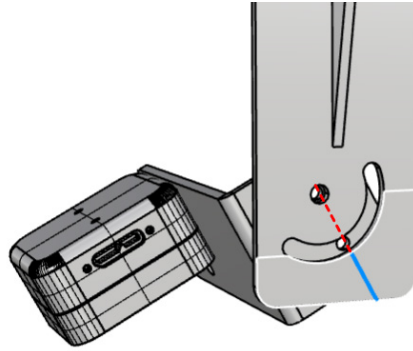
2. 取出下图所示的相机固定件，参考上图，使竖直钣金对齐横杆上的贴纸，使用2颗M6*12螺钉固定。



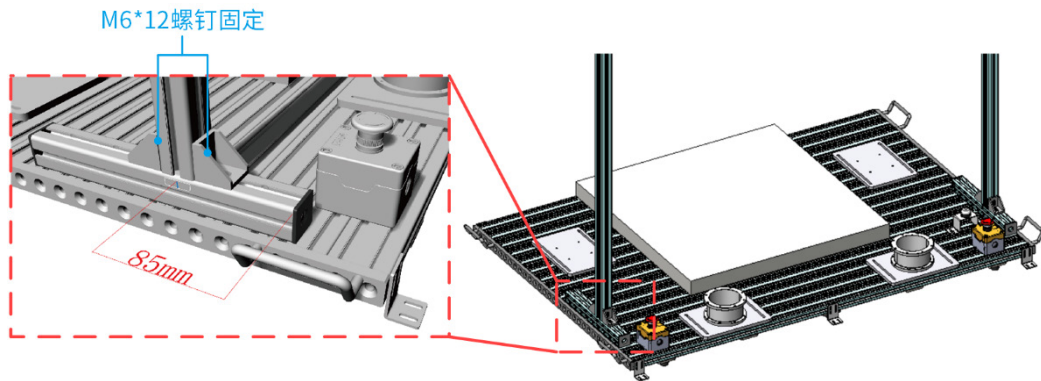
3. 使用1颗M6*8螺钉和3件M6平垫片安装全局相机。



安装完成后，建议参考下图，确认相机支架的角度（两个螺钉孔圆心都位于贴纸上蓝色线条的延长线上）。如未对齐，请手动调整。此处的螺钉规格为M4*10。



4. 立起相机支架，插入到相机支架安装位置的两个L型固定件之间。此时竖杆中线应该会对准下方型材上的贴纸。然后使用2颗M6*12螺钉固定。



5. 参考下图，连接相机线缆。注意相机和主手要接在不同的USB hub上，且相机的USB hub要接在电脑的USB 3.0接口上。请确保相机线缆连接到位，避免因相机接触不良造成后续数据采集异常。



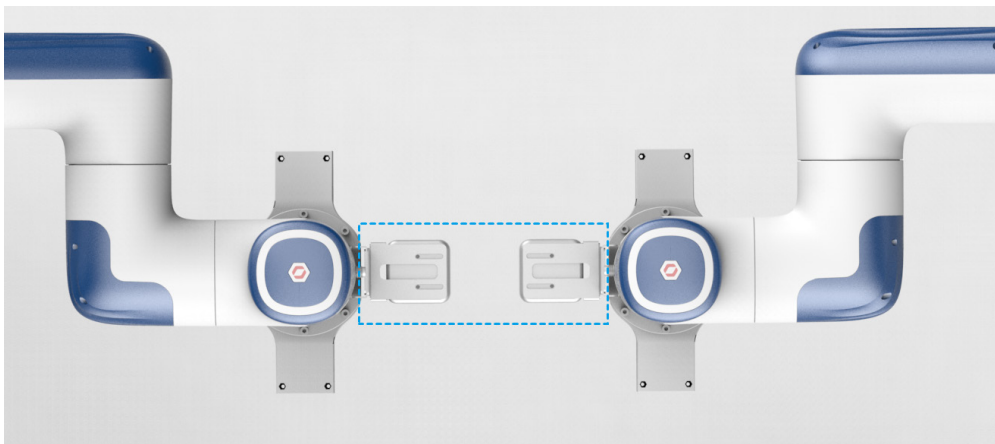
相机连接线如下图所示。



2.3.7 安装从手夹爪及相机

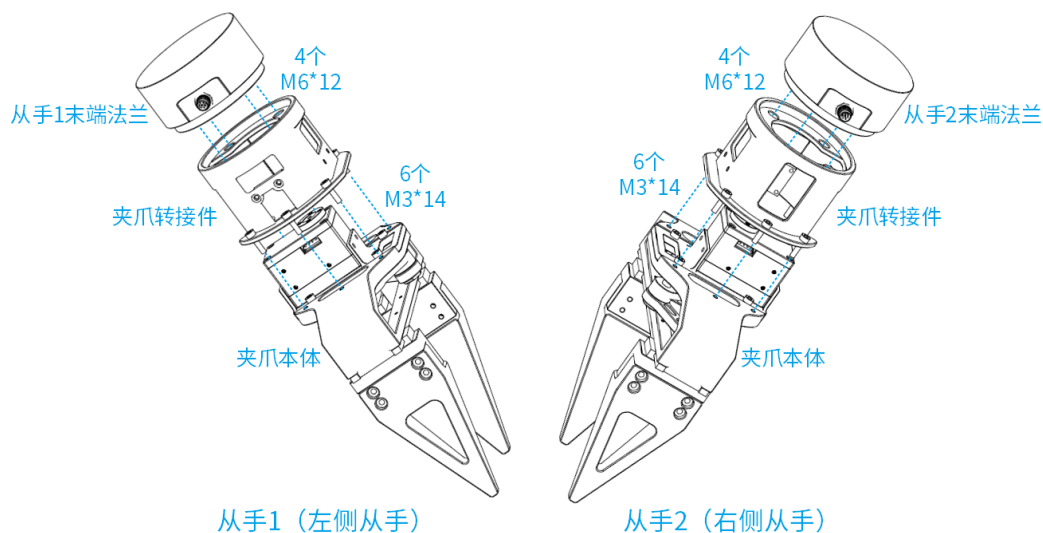
所需工具：内六角扳手*2（M3/M6），M6*12螺钉/M3*6螺钉若干，相机标定卡片*1。

安装从手夹爪和相机前，请先参考[从手上电](#)和[从手设置](#)，将从手1和从手2运动至初始位置，然后再下电，安装夹爪与相机。安装完成后，两个相机应该呈现相对的姿态，如下图所示。

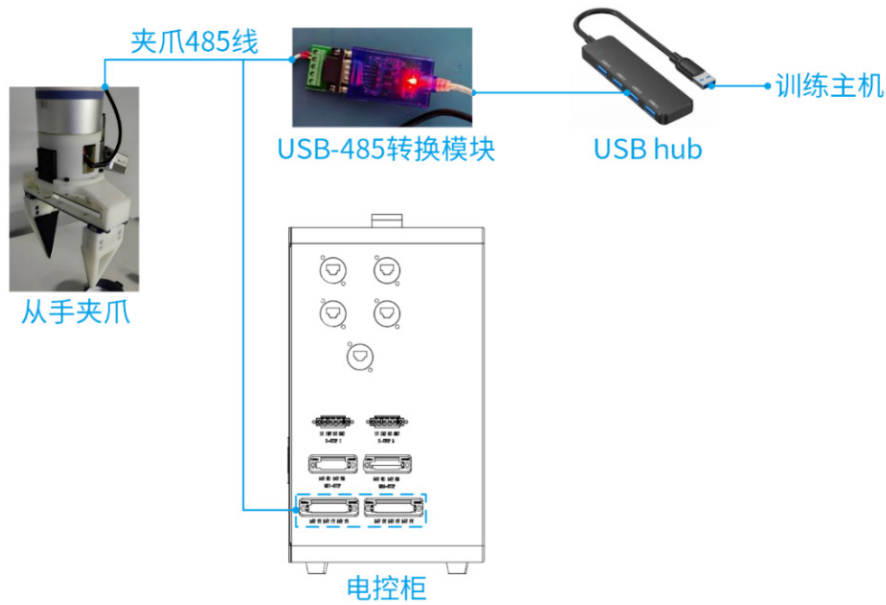


安装从手夹爪

1. 从包装中取出夹爪，拆下6个M3*14螺钉，分离夹爪本体和夹爪转接件。
2. 使用4个M6*12的螺钉将夹爪转接件固定到从手末端法兰。
3. 使用6个M3*14螺钉将夹爪本体固定到夹爪转接件。**两个从手夹爪安装的姿态不同，请参考下图，注意末端法兰航插、夹爪转接件和夹爪本体之间的位置关系。**



4. 参考下图，完成夹爪线缆的连接。



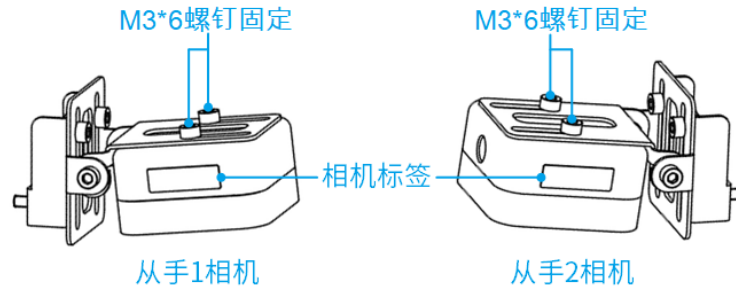
夹爪485线如下图所示，接线说明如下表。



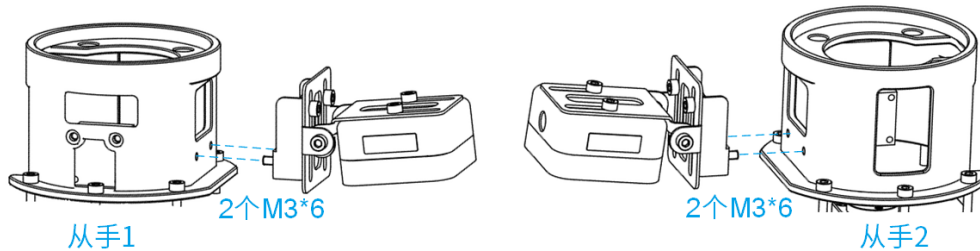
线标（线缆颜色）	对端接口	对端设备
A（绿线）	T/R+	USB-485转换模块
B（白线）	T/R-	
24V（红线）	24V	电控柜
GND（黑线）	0V	
PE（黑线）	悬空	

安装从手相机

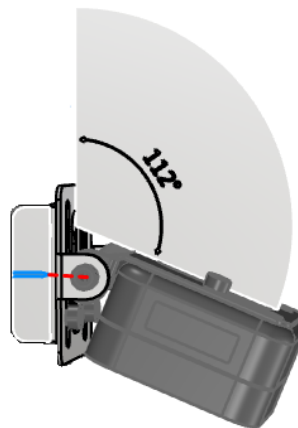
1. 使用2颗M3*6螺钉将相机安装到相机支架上。两个从手的相机需要以相反的姿态安装，请参考下图，确保两个相机相对时标签朝向用户侧。



2. 使用2颗M3*6螺钉将相机支架安装到夹爪转接件上。



安装完成后，建议参考下图进行调整，使支架侧翼螺钉孔对齐贴纸蓝色线条，然后使用发货附件中的扇形标定卡调整角度，使上方夹角为112度，详见下图。



3. 连接相机线缆，连接方法与全局相机相同。注意相机和主手要接在不同的USB hub上，且相机的USB hub要接在电脑的USB 3.0接口上。请确保相机线缆连接到位，避免因相机接触不良造成后续数据采集异常。

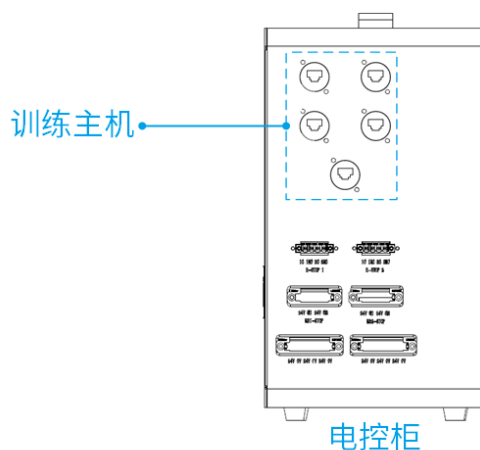
2.3.8 连接指示灯

指示灯出厂时已安装在底座上。使用指示灯连接线连接指示灯和**从手1**控制器。连接控制器时注意对齐线标和控制器上的丝印，接线后线缆应集中在端子左侧。



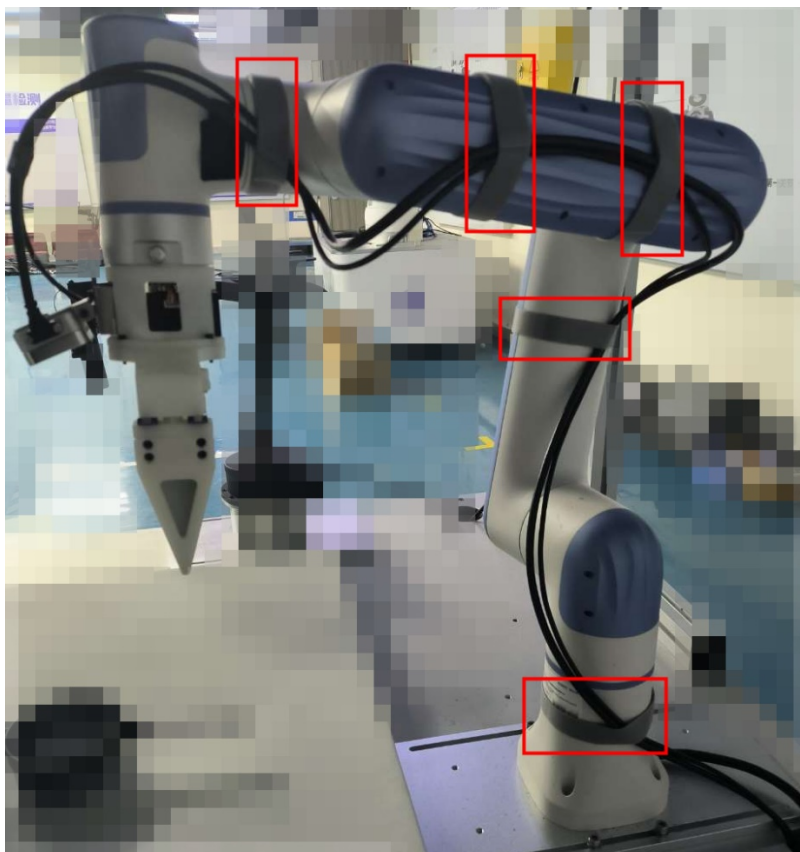
2.3.9 连接训练主机

使用网线连接训练主机和电控柜。



2.3.10 整理线缆

安装接线完成后，建议参考下图，使用发货附件的魔术贴（发货时为一整卷，需用户自行裁剪）将夹爪和相机线缆固定在从手上，避免从手运动时拉扯线缆。



发货附件中还包含一包塑料扎带，用户可用于整理其余线缆。

2.3.11 设备上电

主手上电

主手的电源开关位于底座，按下开关即可上电。上电状态下按下开关可下电。



从手上电

从手的电源开关位于控制器（CCBOX）上，短按开机，长按（3s以上）关机。



从手夹爪上电

从手夹爪由电控柜供电，将电控柜连接至电源即可。电控柜电源接口位于正面（网口等接口的反对面）。



相机上电

相机由训练主机通过USB供电，训练主机开机后相机就会自动上电。

3. 运行环境配置

如无特殊说明，下述操作均在训练主机上执行。若在配置和使用过程中遇到了本文档未说明的问题，请尝试在《Dobot X-Trainer操作指南和FAQ》查找解决方案。

3.1 操作系统

推荐使用Linux系统：Ubuntu 20.04或23.10版本。

说明

- Dobot 未在其他版本 Linux 系统或 Windows 系统下验证过训练与推理环境的配置与使用，无法提供相应的指导。
- 此外，相对 Linux 系统，主从遥操作在 Windows 系统下延迟更高（增加约 10ms）。
- 命名文件夹时请勿使用中文，推荐使用英文字母和数字。

3.2 安装 CUDA 和 cuDNN

说明

如果仅需主从手遥操作功能，可跳过此章。

安装CUDA前请确保已安装最新的显卡驱动。

下文以在Ubuntu 20.04系统中下载和安装CUDA 11.8为例。如您选用的显卡不支持CUDA 11.8，请根据显卡情况安装配套版本的显卡驱动，CUDA，cuDNN和torch。

1. 访问CUDA官网 (<https://developer.nvidia.com/cuda-11-8-0-download-archive>)，根据电脑的操作系统进行选择，获取CUDA runfile的下载和安装指令。



Operating System Linux Windows

Architecture x86_64 ppc64le arm64-sbsa aarch64-jetson

Distribution CentOS Debian Fedora KylinOS OpenSUSE RHEL Rocky SLES Ubuntu

Version 18.04 20.04 22.04

Installer Type deb (local) deb (network) runfile (local)

Download Installer for Linux Ubuntu 20.04 x86_64

The base installer is available for download below.

> Base Installer

Installation Instructions:

```
$ wget https://developer.download.nvidia.com/compute/cuda/11.8.0/local_installers/cuda_11.8.0_520.61.05_linux.run
$ sudo sh cuda_11.8.0_520.61.05_linux.run
```

2. 打开终端，输入上一步获取的下载指令。

```
wget https://developer.download.nvidia.com/compute/cuda/11.8.0/local_installers/cuda_11.8.0_520.61.05_linux.run
```

3. 下载完成后，执行安装指令。

```
sudo sh cuda_11.8.0_520.61.05_linux.run
```

4. 根据界面提示完成安装（因CUDA自带的显卡驱动版本较旧，请在安装CUDA时取消勾选安装显卡驱动的选项，否则可能导致显卡无法正常工作）。
5. 在终端中使用gedit打开用户配置文件（如果未安装gedit，请自行安装并学习gedit的使用方法，也可使用用户常用的文本编辑器，比如vim）。

```
sudo gedit ~/.bashrc
```

6. 在文件中添加以下内容并保存。下方示例为默认安装路径，如果修改过CUDA的安装路径，请自行修改指令中的路径。

```
export PATH=/usr/local/cuda-11.8/bin:$PATH
```

```
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/usr/local/cuda-11.8/lib64
```

7. 重启操作系统。
8. 验证CUDA是否安装成功。执行命令后输出了nvcc的版本信息即为安装成功，否则请重新安装。

```
nvcc -V
```

9. 访问cuDNN官网 (<https://developer.nvidia.com/cudnn-downloads>)，根据电脑的操作系统进行选择，获取cuDNN的下载和安装指令。

The screenshot shows the NVIDIA cuDNN download page. Under 'Operating System', 'Linux' is selected. Under 'Architecture', 'x86_64' is selected. Under 'Distribution', 'Ubuntu' is selected. Under 'Version', '20.04' is selected. Under 'Installer Type', 'deb (local)' is selected. The page title is 'Download Installer for Linux Ubuntu 20.04 x86_64'. Below this, it says 'The base installer is available for download below.' and shows a 'Download (1.6 GB)' button. A red box highlights the 'Installation Instructions' section, which contains the following commands:

```
$ wget https://developer.download.nvidia.com/compute/cudnn/9.1.1/local_installers/cudnn-local-repo-ubuntu2004-9.1.1_1.0-1_amd64.deb
$ sudo dpkg -i cudnn-local-repo-ubuntu2004-9.1.1_1.0-1_amd64.deb
$ sudo cp /var/cudnn-local-repo-ubuntu2004-9.1.1/cudnn-*-keyring.gpg /usr/share/keyrings/
$ sudo apt-get update
$ sudo apt-get -y install cudnn
```

Below this, it says 'To install for CUDA 11, perform the above configuration but install the CUDA 11 specific package:' and shows the command:

```
$ sudo apt-get -y install cudnn-cuda-11
```

10. 参考官网提供的指令，完成cuDNN的下载与安装。

3.3 安装 Anaconda

1. 从官网 (<https://www.anaconda.com/download#download-section>) 或者镜像站 (<https://mirrors.tuna.tsinghua.edu.cn/help/anaconda/>) 下载 Anaconda 安装

包。

2. 进入下载目录，打开终端，运行安装包（指令仅为示例，请以实际的sh文件名为准）。

```
bash Anaconda3-2020.07-Linux-x86_64.sh
```

3. 根据界面提示完成安装。

4. 在终端中使用gedit打开用户配置文件。

```
sudo gedit ~/.bashrc
```

5. 在文件中添加以下内容并保存。请自行修改指令中的路径为anaconda实际的安装路径。

```
export PATH="/home/dobot/anaconda3/bin:$PATH"
```

6. 在终端中执行下述指令，应用环境变量。

```
source ~/.bashrc
```

7. 验证Anaconda是否安装成功。执行命令后输出了conda的版本信息即为安装成功，否则请重新安装。

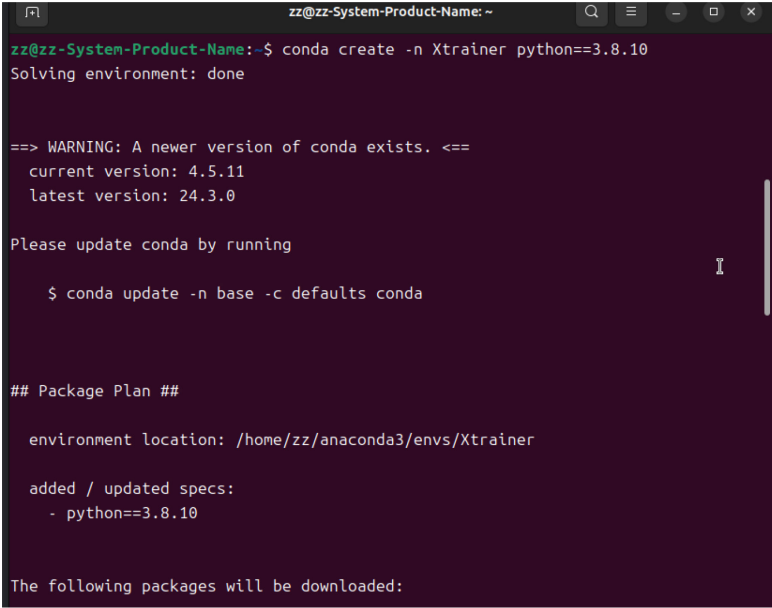
```
conda -V
```

3.4 配置虚拟环境

1. 在终端命令窗口或Anaconda命令窗口新建一个虚拟环境并安装python3.8.10。

例如创建一个名为“Xtrainer”的虚拟环境：

```
conda create -n Xtrainer python==3.8.10
```



```
zz@zz-System-Product-Name: ~  
zz@zz-System-Product-Name:~$ conda create -n Xtrainer python==3.8.10  
Solving environment: done  
  
==> WARNING: A newer version of conda exists. <==  
current version: 4.5.11  
latest version: 24.3.0  
  
Please update conda by running  
  
$ conda update -n base -c defaults conda  
  
## Package Plan ##  
  
environment location: /home/zz/anaconda3/envs/Xtrainer  
  
added / updated specs:  
- python==3.8.10  
  
The following packages will be downloaded:
```

输入“y”确认下载。

```

zz@zz-System-Product-Name: ~
Proceed ([y]/n)? y

Downloading and Extracting Packages
readline-8.1.2 | 291 KB | ##### | 100%
ncurses-6.3 | 1002 KB | ##### | 100%
setuptools-65.5.1 | 731 KB | ##### | 100%
libgcc-ng-12.2.0 | 931 KB | ##### | 100%
ld_impl_linux-64-2.3 | 759 KB | ##### | 100%
python-3.8.10 | 26.1 MB | ##### | 100%
libffi-3.4.2 | 57 KB | ##### | 100%
libgomp-12.2.0 | 455 KB | ##### | 100%
libsqlite-3.40.0 | 791 KB | ##### | 100%
_libgcc_mutex-0.1 | 3 KB | ##### | 100%
openssl-1.1.1s | 2.1 MB | ##### | 100%
libzlib-1.2.13 | 64 KB | ##### | 100%
ca-certificates-2022 | 150 KB | ##### | 100%
wheel-0.38.4 | 32 KB | ##### | 100%
tk-8.6.12 | 3.3 MB | ##### | 100%
xz-5.2.6 | 409 KB | ##### | 100%
libstdc++-12.2.0 | 4.3 MB | ##### | 100%
sqlite-3.40.0 | 801 KB | ##### | 100%
_openmp_mutex-4.5 | 23 KB | ##### | 100%
zlib-1.2.13 | 92 KB | ##### | 100%
pip-22.3.1 | 1.5 MB | ##### | 100%
Preparing transaction: done
Verifying transaction: done
Executing transaction: done
#

```

2. 激活并进入虚拟环境，执行命令后显示(Xtrainer)则表明已进入虚拟环境。

```
conda activate Xtrainer
```

```

zz@zz-System-Product-Name: ~
zz@zz-System-Product-Name:~$ conda activate Xtrainer
(Xtrainer) zz@zz-System-Product-Name:~$

```

3. 安装pytorch 2.0.1和torchvision 0.15.2。

说明

如果仅需主从手遥操作功能，可跳过此步骤。

```
pip install torch==2.0.1 torchvision==0.15.2 torchaudio==2.0.2 --index-url https://download.pytorch.org/whl/cu118
```

安装完成后，依次执行下列指令验证torch/torchvision是否安装成功。

```
python
import torch
import torchvision
torch.cuda.is_available()
exit()
```

执行torch.cuda.is_available()后返回True表示安装成功，否则请检查CUDA和cuDNN是否已安装，torch/torchvision是否安装了正确的版本。如果安装了错误版本的torch/torchvision，请使用pip uninstall指令卸载错误的版本后重新安装。

4. 在Dobot_Xtrainer（越疆提供的X-Trainer示例工程）文件夹内打开终端，进入虚拟环境（参见步骤2），依次执行下列指令安装依赖包。

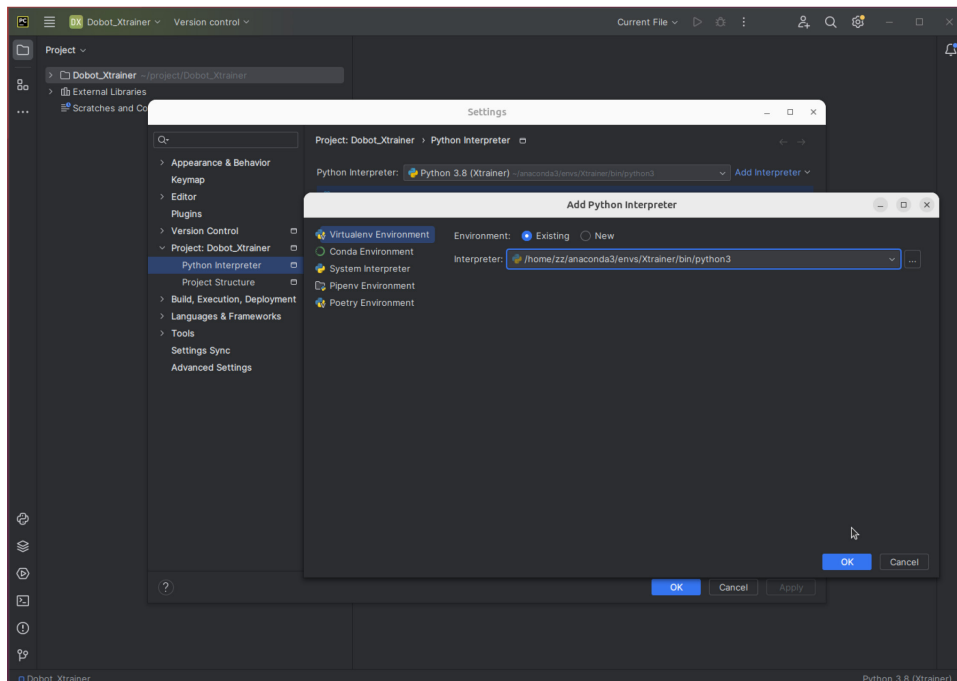
```
pip install -r requirements.txt
pip install -e third_party/DynamixelSDK/python
```

说明

如果仅需主从手遥操作功能，无需执行下面三条指令。

```
pip install -e ModelTrain/detr
pip install -e robomimic-r2d2
sudo apt install libxcb-cursor0
```

5. 在IDE中设置使用虚拟环境（Xtrainer）。以PyCharm为例：通过PyCharm打开Dobot_Xtrainer文件夹，依次打开Pycharm的**Files > Settings > Python Interpreter > Add Interpreter > Existing > xxxx/anaconda3/envs/Xtrainer/bin/python3**（xxx为anaconda的实际安装路径），然后单击**OK**和**Apply**完成设置。

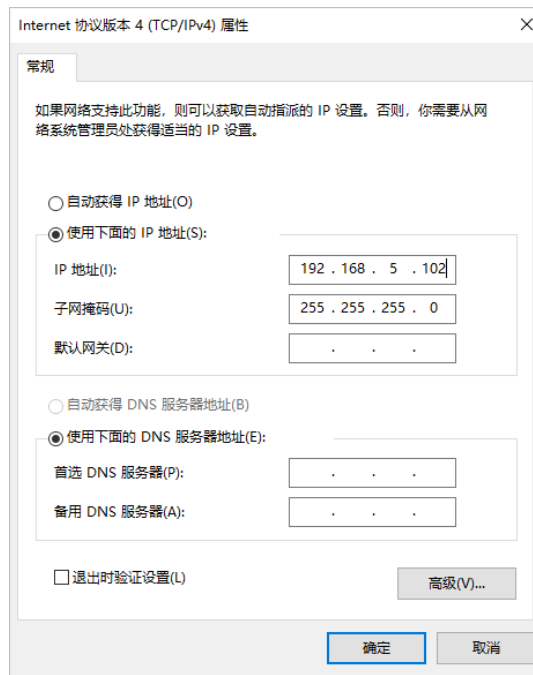


3.5 从手设置

3.5.1 修改从手 2 的 IP

从手1和从手2的LAN1网口默认IP相同（都为192.158.5.1），因此需要将右手（右侧从手）的IP修改为192.168.5.2，避免冲突。修改步骤如下，具体操作可参考DobotStudio Pro的帮助文档。

1. 使用网线连接从手2的CCBOX和**从手操作终端（以电脑为例）**。
2. 请将电脑用于连接从手的网卡IP修改为192.168.5.xxx网段（子网掩码255.255.255.0），且不与两台从手的IP冲突，例如192.168.5.102。

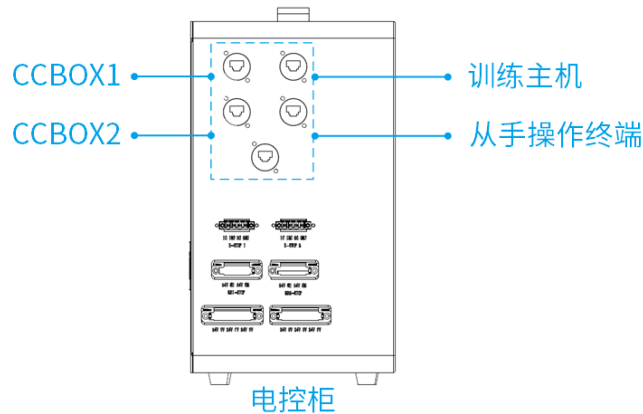


3. 在电脑上打开从手的控制软件（DobotStudio Pro 2.x.x），连接搜索到的Nova 2。
4. 打开**设置 > 通讯设置**，在**IP设置**标题下选择**手动设置IP**，修改IP为192.168.5.2。



5. 单击**应用**。

如果连接时使用了CCBOX的LAN1网口，请在配置完成后重新将LAN1网口通过网线连接到电控柜，然后将**从手操作终端**也接至电控柜，如下图所示。



3.5.2 配置从手 1 和 2

请针对从手1和从手2分别执行下述操作。

1. 在DobotStudio Pro中手动添加两台从手的IP。



2. 使用DobotStudio Pro连接从手。
 - 从手1（左侧从手）：Nova 2（192.168.5.1）
 - 从手2（右侧从手）：Nova 2（192.168.5.2）
3. 单击界面右上的使能开关，将负载设置为0.5kg。



4. 设置从手的初始姿态。

- a) 打开设置 > 基础设置页面，单击初始姿态右侧的重新设置初始点位。



- b) 将初始姿态设为下述的值（6个值分别表示J1~J6的关节角度）并确定。

从手1: {-90, 0, -90, 0, 90, 90}

从手2: {90, 0, 90, 0, -90, -90}



- c) 长按移动至初始点位，直至从手运动到位。



⚠ 注意

从手运动过程中请注意观察，如果可能发生碰撞，请及时松开按钮，通过点动等方法避开障碍后再继续移动至初始点位。

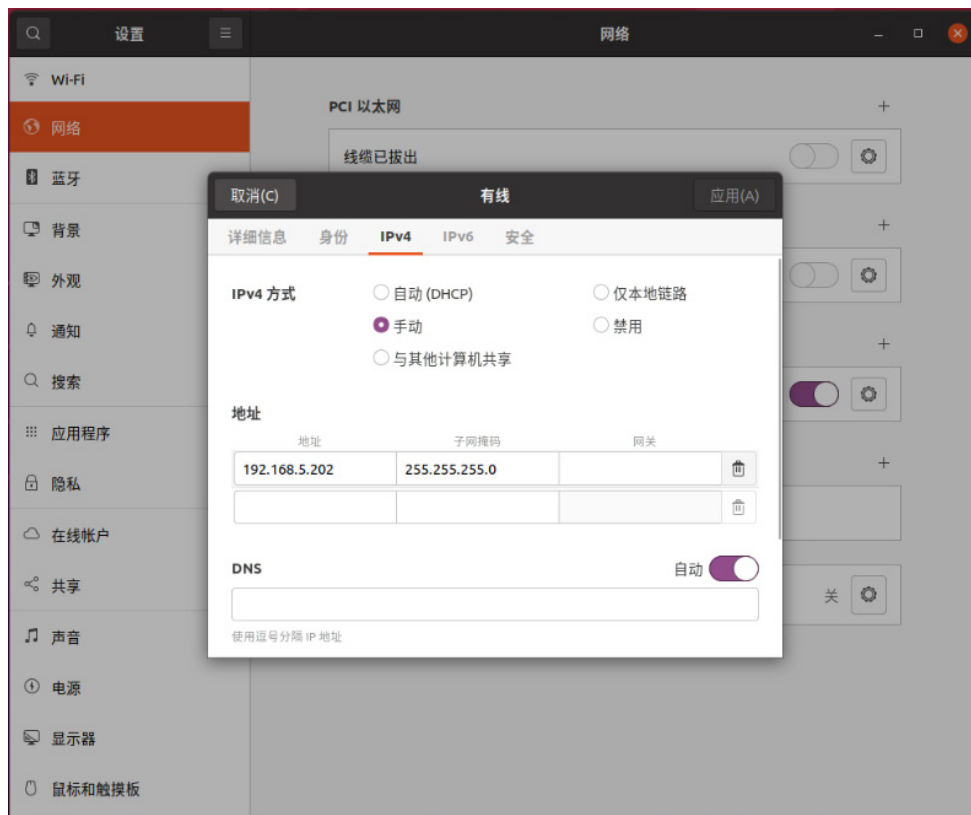
5. 打开设置 > 远程控制页面，将当前模式设置为TCP/IP二次开发，然后单击应用。



完成上述配置后，后续操作均在**训练主机**上进行，但从手如果出现报错等异常状态，需要使用**从手操作终端**打开DobotStudio Pro进行处理。

3.6 配置训练主机 IP

请将训练主机用于连接电控柜的网卡 IP 修改为 192.168.5.xxx 网段（子网掩码 255.255.255.0），且不与两台从手以及从手操作终端的 IP 冲突，例如 192.168.5.202（下图以 Ubuntu 的设置界面为例）。配置后请打开对应有线网络的开关。



4. 遥操作与数据采集

4.1 主手操作说明

本节介绍了主手末端手柄的使用方法，请根据后文完成配置后，参考本节操作机器人。



- 解锁/同步键（左侧黄色按键）：
 - 短按（0.5s）锁定/解锁主手，解锁时可自由拖动主手。
 - 长按（2s）同步/断开同步。同步时从手以低速同步到主手当前的位姿和正常操作速度，同步完成后即可进行遥操作控制从手的运动。左右主手需要分别使用该按键进行解锁和同步。
- 录制键（右侧绿色按键）：

短按（0.5s）开始录制并采集数据，再次短按终止录制。

主从手同步成功后，按下任一侧的该按键都会开始或终止X-Trainer整体的数据录制，不支持单独录制某一侧的数据。
- 夹爪控制扳机：

按下扳机从手夹爪闭合，松开扳机从手夹爪张开。可根据按下的距离控制夹爪的开合度。

⚠ 注意

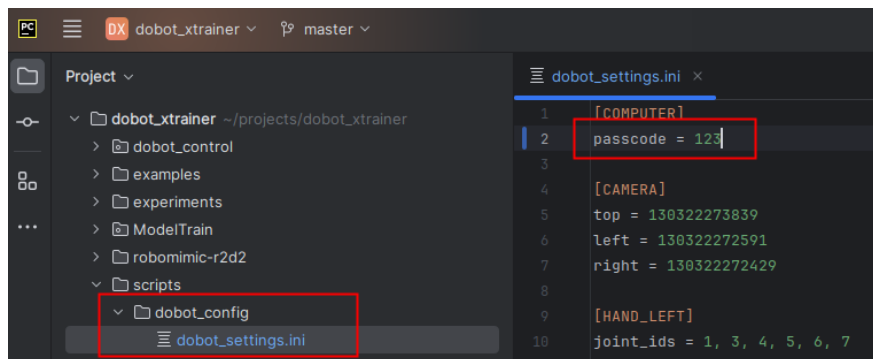
- 请双手握住手柄后再解锁，避免主、从手的意外掉落。
- 请确保主手之间保持适当的安全距离，并锁定到安全位置后再同步主从机械臂，避免从手出现意外掉落、意外碰撞等异常。
- 锁定后请勿碰撞主手，避免舵机损坏。
- 操作时注意避免从手相互干涉导致损坏机械臂或者相机。

4.2 配置并采集数据

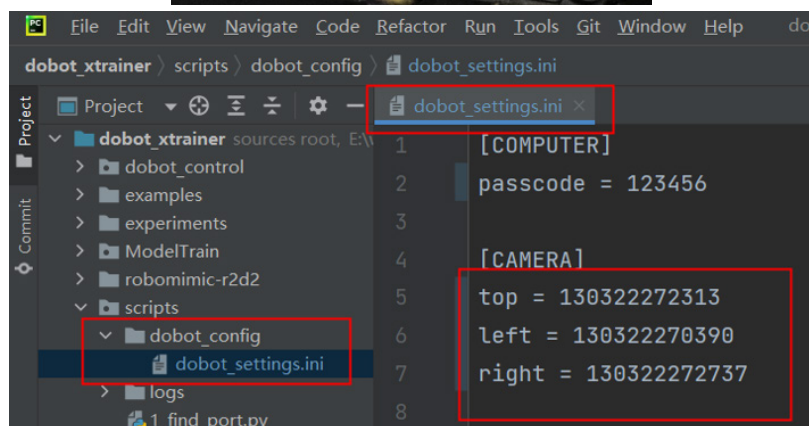
1. 在IDE中（以PyCharm为例）打开Dobot_Xtrainer文件夹。

2. 修改 `scripts > dobot_config > dobot_settings.ini` 文件。

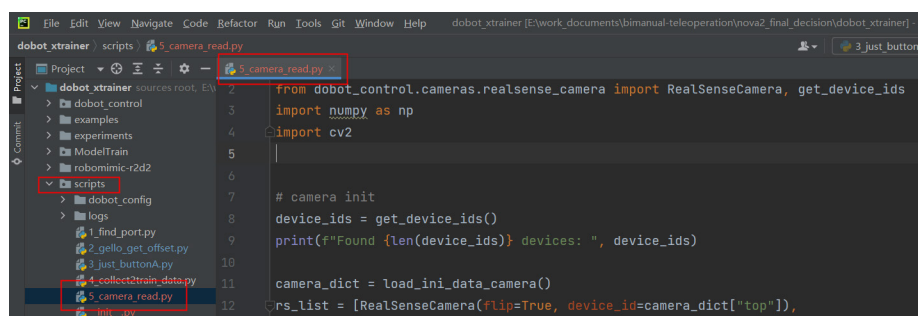
- a) 修改密码为本机密码，用于执行 `sudo` 指令。



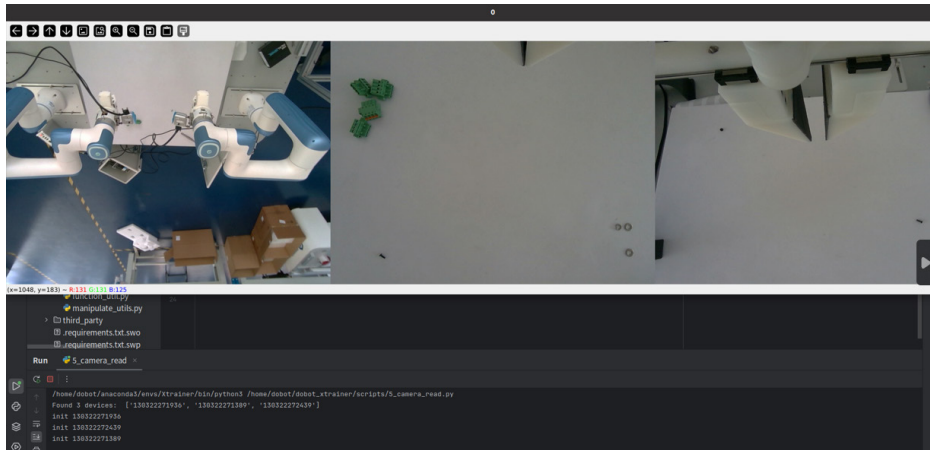
- b) 填写各个相机的 ID。top 表示全局相机，left 表示从手 1 相机，right 表示从手 2 相机。



保存配置后建议运行 `scripts > 5_camera_read.py` 脚本，查看程序是否能正确读取到相机图像。



正常运行的效果如下图。

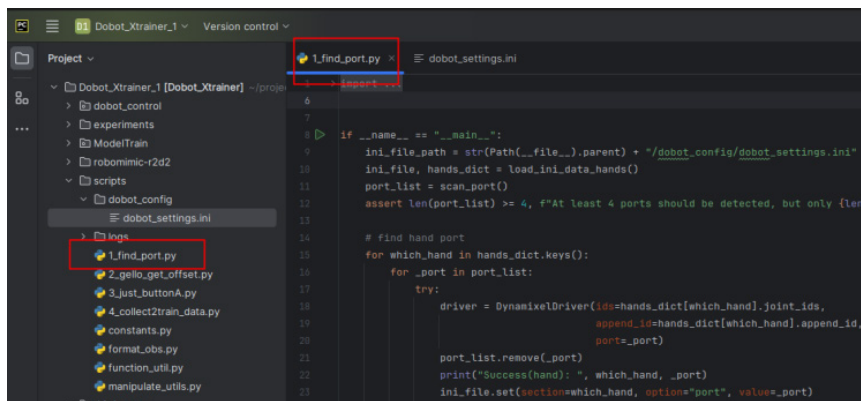


如果图像异常或脚本报错，请重新插拔相机数据线，并检查各相机 ID 是否正确填写，然后再重新运行脚本。

相机测试完成后请手动停止脚本运行。

3. 运行scripts > 1_find_port.py脚本，设置两个主手和两个从手夹爪的USB接口。

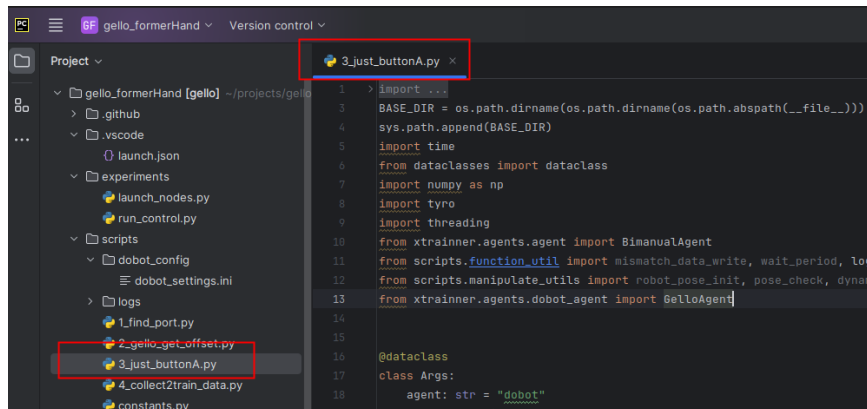
每次训练主机断电或拔插USB接口后都需要重新运行该脚本。



4. 设置主从手同步姿态。

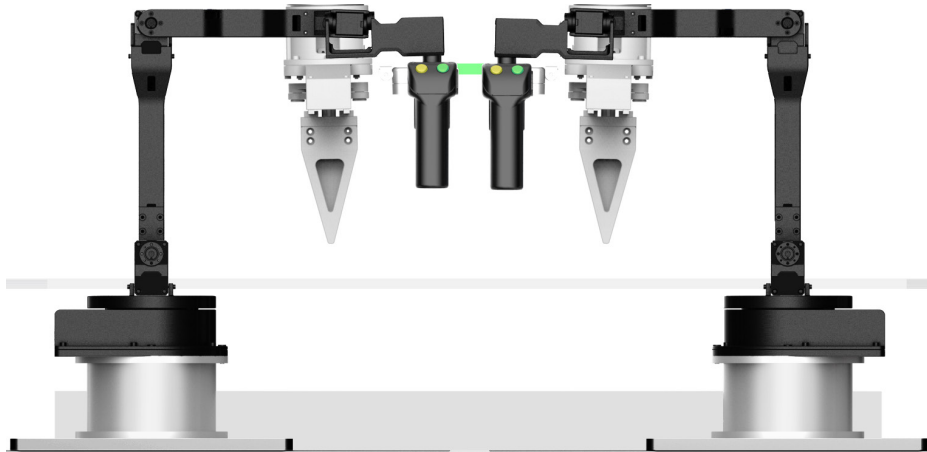
完成该步骤后主从手的同步姿态信息会保存在dobot_settings.ini中，后续如无异常（主从手不同步或配置文件丢失等）无需再重复设置。

- a) 运行scripts > 3_just_buttonA.py脚本，运行后可解锁并拖动主手。

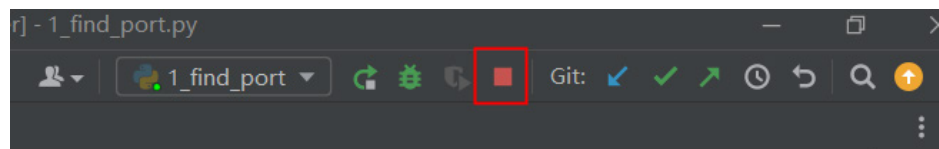


- b) 分别握住左右主手的末端手柄并短按黄色按键解锁主手，拖动两个主手，直至

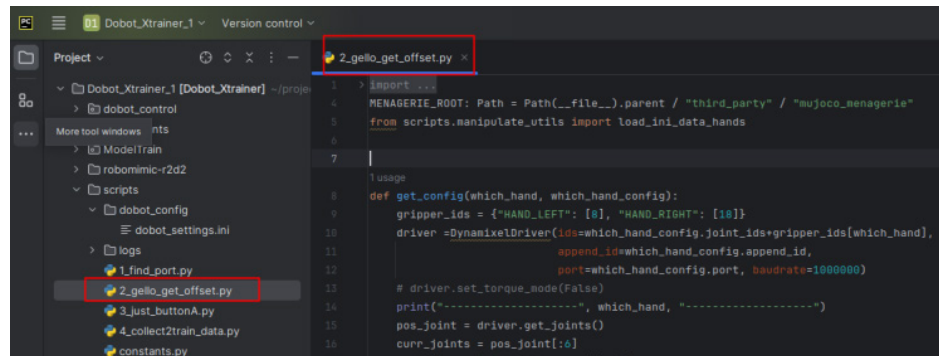
主手姿态与从手的初始姿态（参见3.5.2 配置从手1和2）相同，再次短按两个主手的黄色按键锁定主手。



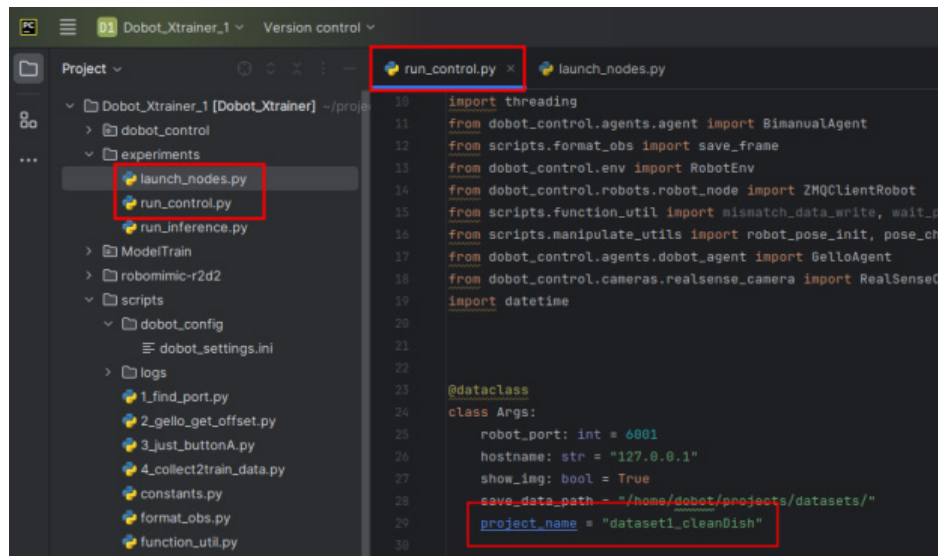
- c) 停止`3_just_buttonA.py`脚本。执行下一步前请确保脚本已完全停止，以PyCharm为例，如果第一次点击停止按钮后该按钮未消失，则需要再次点击停止按钮，待停止按钮消失后才表示脚本已停止。



- d) 运行`scripts>2_gello_get_offset.py`脚本，将主手和从手的当前姿态设置为同步的基准值。



5. 运行`experiments>launch_nodes.py`脚本，开启从手的服务端进程。
6. 打开`experiments>run_control.py`脚本，根据实际要训练的任务自定义数据集名称（`project_name`），然后保存并运行脚本。
- 运行上述两个脚本后可开始遥操作和训练数据的录制。



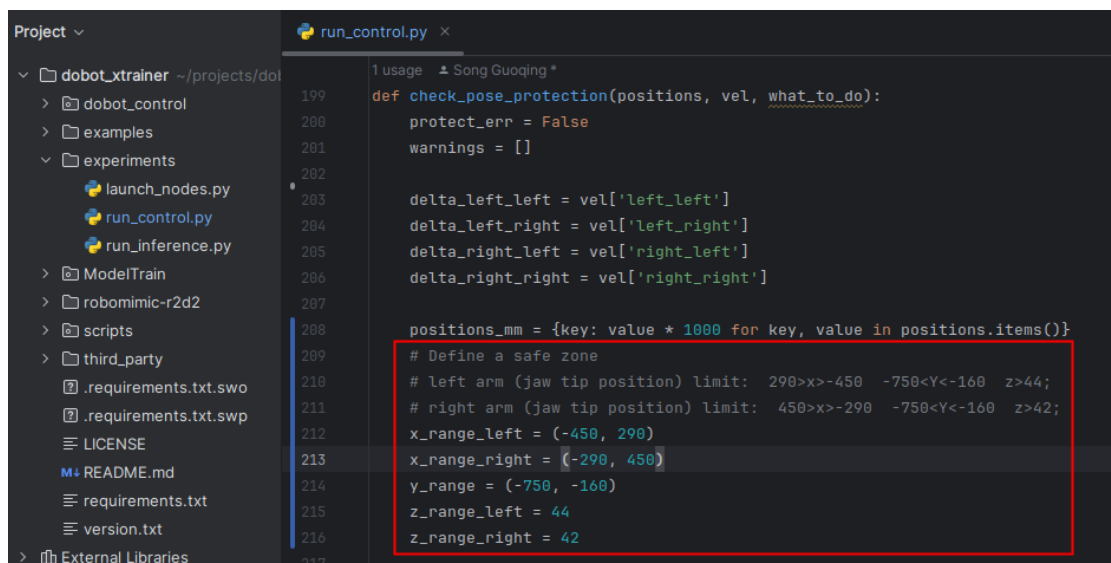
出于安全考虑，越疆针对遥操作设置了一些速度和操作位置相关的安全限制，触发这些限制会导致遥操作停止且台面上的指示灯亮红灯报警。此时用户需要停止再重新运行 `run_control.py` 脚本解除报警。

⚠ 注意

重新运行脚本后，建议：

1. 先短按黄色按键解锁主手并将其拖动至初始位置并锁定。
2. 再长按黄色按键同步从手姿态。
3. 然后解锁主手进行遥操作。

默认情况下，从手夹爪只能在工作台上的EVA泡棉正上方区域活动。如有需要，可修改 `run_control.py` 脚本中的从手活动区域限制值（如下图），但修改前必须评估修改可能带来的安全风险。



5. 模仿学习算法训练

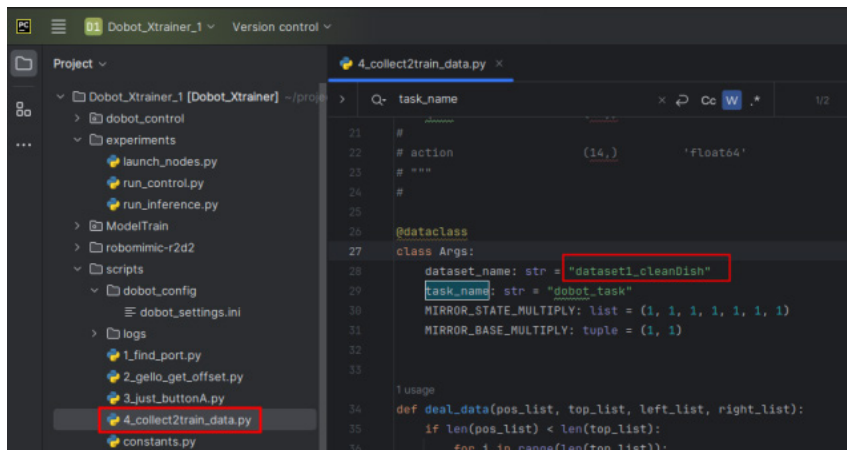
5.1 训练数据集处理

采集完成后，可在与Dobot_XTrainer同级的datasets文件夹下找到与数据集同名（即experiments>run_control.py中的project_name）的文件夹（后文简称为训练数据文件夹）。



该文件夹内此时应该只有一个collect_data文件夹，里面保存了原始采集数据。如果需要使用采集的数据进行算法训练，需要先对数据进行压缩整合，步骤如下。

1. 打开scripts>4_collect2train_data.py脚本，将数据集名称（dataset_name）修改为实际录制的数据集名称。



2. 运行脚本，对训练数据进行压缩整合。
3. 脚本运行完成后，训练数据文件夹内会生成两个新文件夹：output_videos和train_data。
 - i. output_videos是训练过程中相机采集的视频，可用于查看采集过程操作有无异常。
 - ii. train_data是处理后的训练数据，用于后续训练步骤。

5.2 设置数据集相关参数

打开ModelTrain > contants.py脚本，修改如下参数：

```
DATA_DIR = '~/projects/datasets/dataset1_cleanDish' ①
TASK_CONFIGS = {
    # dobot move cube new
    ② 'move_cube_new': {
        'dataset_dir': DATA_DIR + '/train_data', ③
        'episode_len': 900, ④
        'train_ratio': 0.9, ⑤
        'camera_names': ['top', 'left_wrist', 'right_wrist'] ⑥
    },
}
```

- ①：数据集所在文件夹的路径，即前述的train_data文件夹的上级目录。
- ②：数据集名称。
- ③：固定填写为/train_data，使①+③等于上一节得到的train_data文件夹的完整路径。
- ④：采集步数。可通过运行scripts>6_Dataset_Count.py脚本获取数据集的最大步数，然后设置为比该值略大的值，例如数据集的最大步数为410，则此参数可设置为500。

```
dataset_name
class Args:
    dataset_name: str = "dataset_package_test" a
def main(args):
    root_dir = str(Path(__file__).parent.parent.parent / "datasets/")
    dataset_dir = root_dir + "/" + args.dataset_name + "/collect_data/"
    all_data_dir = os.listdir(dataset_dir)
    all_data_dir.sort(key=lambda x: int(x))
    max_step = 0
    for idx in range(len(all_data_dir)):
        one_data_dir = dataset_dir+all_data_dir[idx]+"/"
        data_pose_list = glob.glob(one_data_dir + 'observation/*.pkl')
        if len(data_pose_list) > max_step:
            max_step = len(data_pose_list)
    print("max step: ", max_step)
main()
```

```
Run: 6_Dataset_Count
F:\work_software\Anaconda3\envs\aloha\pythonw.exe E:/work_documents/bimanual-teleoperation/dobot
max step: 0 b
Process finished with exit code 0
```

- a：修改为实际的数据集名称后运行脚本。
 - b：运行后得到的该数据集的最大步数。
 - ⑤：训练集的比例，其余数据会作为验证集。
 - ⑥：相机名称，如未修改过相机相关代码，请保持默认值。
- 修改完成后保存。

5.3 设置训练相关参数

打开ModelTrain > model_train.py脚本，修改如下参数（修改对应行的default参数取值）：

```
def arg_config():
    parser = argparse.ArgumentParser()
    parser.add_argument('--ckpt_dir', action='store', type=str, help='ckpt_dir', default='./ckpt/ckpt_move_cube_new', required=False)
    parser.add_argument('--task_name', action='store', type=str, default='move_cube_new', help='task_name', required=False)
    parser.add_argument('--batch_size', action='store', type=int, help='batch_size', default=16, required=False)
    parser.add_argument('--seed', action='store', type=int, help='seed', default=0, required=False)
    parser.add_argument('--num_steps', action='store', type=int, help='num_steps', default=200, required=False)
    parser.add_argument('--lr', action='store', type=float, help='lr', default=2e-5, required=False)
    parser.add_argument('--load_pretrain', action='store_true', default=False) # Ignore this parameter and leave the default setting
    parser.add_argument('--eval_every', action='store', type=int, default=100, help='eval_every', required=False) # Ignore this parameter and leave the def
    parser.add_argument('--validate_every', action='store', type=int, default=100, help='validate_every', required=False)
    parser.add_argument('--save_every', action='store', type=int, default=1000, help='save_every', required=False)
    # parser.add_argument('--resume_ckpt_path', action='store', type=str, help='resume_ckpt_path', default='./ckpt/ckpt_closh/ckpt_closh_puping2_new10000/policy_last.ck
    parser.add_argument('--resume_ckpt_path', action='store', type=str, help='resume_ckpt_path', required=False)
    parser.add_argument('--skip_mirrored_data', action='store_true')

    parser.add_argument('--kl_weight', action='store', type=int, help='KL divergence weight, recommended set 10 or 100', default=10, required=False)
    parser.add_argument('--chunk_size', action='store', type=int, help='The model predicts the length of the output action sequence at a time', default=45, r
    parser.add_argument('--hidden_dim', action='store', type=int, help='hidden_dim', default=512, required=False)
    parser.add_argument('--dim_feedforward', action='store', type=int, help='dim_feedforward', default=3200, required=False)
    parser.add_argument('--temporal_agg', action='store_true', default=True)
    parser.add_argument('--no_encoder', action='store_true', default=False)

    return vars(parser.parse_args())
```

- ckpt_dir：设置训练后得到的模型要保存的路径，可自定义。
- task_name：数据集名称。
- batch_size：训练时 batch 的大小，根据显卡性能设置（例如单张 RTX4090 可设置为 16）。
- num_steps：训练迭代次数。
- lr：学习率。
- validate_every：每训练迭代多少次时验证一次。
- save_every：每训练迭代多少次时保存一次模型。
- resume_ckpt_path：如果需要基于已保存的模型继续训练，请参考上一行被注释掉的代码，添加 default 参数，取值为已保存的模型地址。
- chunk_size：算法一次预测的动作序列长度。
- temporal_agg：动作时间平滑，建议设置为 True。
- 其他参数一般保持默认即可

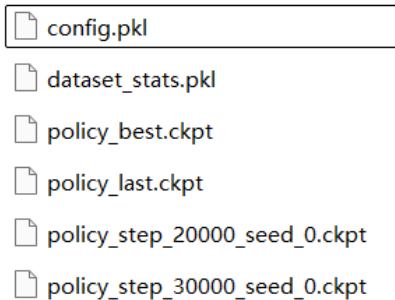
修改完成后，保存并运行脚本即可开始训练，如下图所示，请耐心等待训练完成。第一次训练时，需要联网下载预训练模型，后续训练时则不再需要联网。

```
Run model_train x launch_nodes x run_inference x
Val loss: 0.25279
val_l1: 0.253 val_kl: 0.000 val_loss: 0.253
88% | 165700/188889 [15:31:12<1:47:48, 3.58it/s] validating
Val loss: 0.25166
val_l1: 0.252 val_kl: 0.000 val_loss: 0.252
88% | 165800/188889 [15:31:46<1:47:46, 3.57it/s] validating
Val loss: 0.25459
val_l1: 0.255 val_kl: 0.000 val_loss: 0.255
88% | 165900/188889 [15:32:19<1:46:53, 3.58it/s] validating
Val loss: 0.24766
val_l1: 0.248 val_kl: 0.000 val_loss: 0.248
88% | 166000/188889 [15:32:53<1:46:31, 3.58it/s] validating
Val loss: 0.25496
val_l1: 0.255 val_kl: 0.000 val_loss: 0.255
88% | 166082/188889 [15:33:22<1:46:28, 3.57it/s]
```

训练完成后，控制台显示“Train Finished”并弹出曲线图展示训练结果，如下图所示。



训练好的模型保存在前述的`ckpt_dir`参数指定的文件夹内，里面包含多个模型，自助推理运行一般使用`policy_last.ckpt`（最后的训练模型）或`policy_best.ckpt`（验证精度最好的模型）。



可打开**ModelTrain > model_inference_test.py**，可通过输入采集的数据集的图片 and 从手关节值（包括夹爪开合值）到训练好的模型中，测试模型推理输出的从手关节和夹爪的动作值，和数据标签真值进行对比，从而初步判断模型预测准确度情况。

6. 自主推理运行

1. 打开 `experiments > run_inference.py` 脚本，修改下述参数。

```
# Initialize the model
model_name = 'policy_last.ckpt'
# model_name = 'policy_best.ckpt'
model = Imitate_Model(ckpt_dir='./ckpt/ckpt_move_cube_new', ckpt_name=model_name)

episode_len = 900 # 完成任务总步长
```

- `model_name`: 模型名称。设置为 `policy_last.ckpt`（最后的训练模型）或 `policy_best.ckpt`（验证精度最好的模型）。
- `ckpt_dir`: 模型所在地址，参见 `ModelTrain > model_train.py` 中的同名参数。
- `episode_len`: 任务步长，必须小于等于 `ModelTrain > contents.py` 中同名参数的取值。用户可通过修改该参数裁剪自主推理运行的动作序列长度，例如当自主推理运行时机器人未做完目标动作，可增加步长；如果机器人做完目标动作后还有些多余的动作，则可减少步长。

修改完成后保存。

2. 运行 `experiments > launch_nodes.py` 脚本，开启从手的服务端进程。
3. 运行 `experiments > run_inference.py` 脚本，从手开始基于推理自主运行。

注意

运行 `run_inference.py` 时必须保证 `model_train.py` 中的参数与训练该模型时相同。

7. 数据接口说明

越疆提供了一套的DEMO用于演示如何获取Dobot XTrainer的数据，位于Dobot XTrainer工程的example文件夹内：

- example_agent.py：获取主手关节角和按键信息的示例。
- example_camera.py：获取相机图像的示例。
- example_dobot_robot.py：设置指示灯和获取从手关节角的示例。
- example_gripper.py：控制从手夹爪的示例。
- example_read_data_from_dataset.py：解析数据集中的数据的示例。